E-COMMERCE WEBSITE PROJECT

The project has two versions:

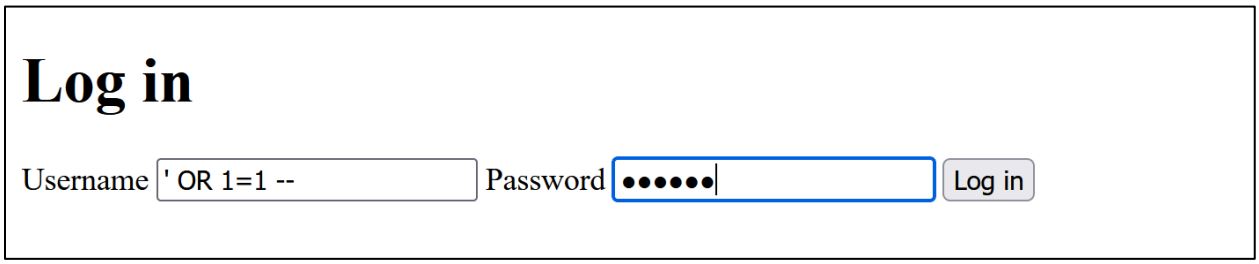
Vulnerable version: contains intentionally insecure code that allows the vulnerabilities to be demonstrated.

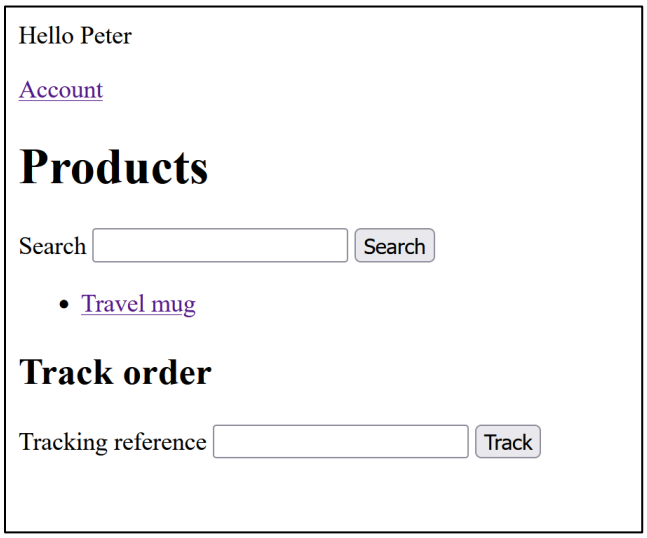
Secure version: keeps the same basic functionality but applies security measures to prevent the vulnerabilities.

1. SQL Injection:

Is a vulnerability where an attacker can manipulate an SQL query by providing malicious input.

In this application, the login function is vulnerable.



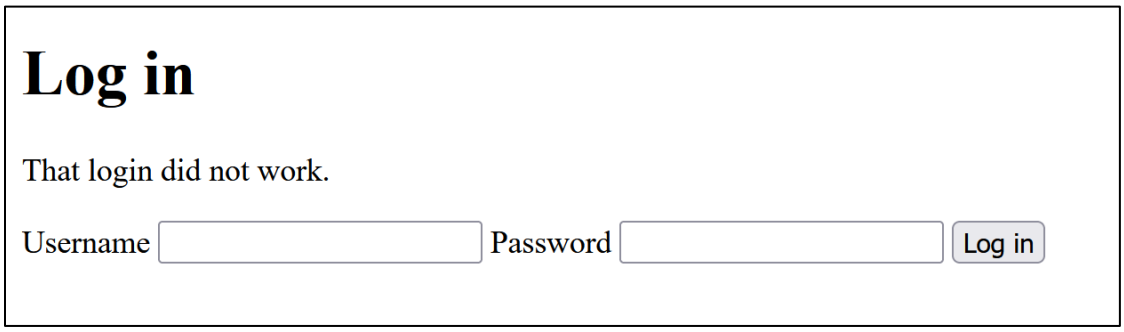


Problem: user input is directly concatenated into the SQL query, which allows the input to affect the SQL query itself.

```
@app.route("/login", methods=["GET", "POST"])
def login():
    error = None
    next_url = request.values.get("next", url_for("products"))
    if request.method == "POST":
        username = request.form.get("username", "")
        password = request.form.get("password", "")
        query = (
            "SELECT username FROM accounts WHERE username = '"
            + username
            + "' AND password = '"
            + password
            + "' LIMIT 1"
        )
```

Fix: modifying the code by using “?” as placeholders to ensure that user input is treated as data rather than SQL code, preventing SQL injection.

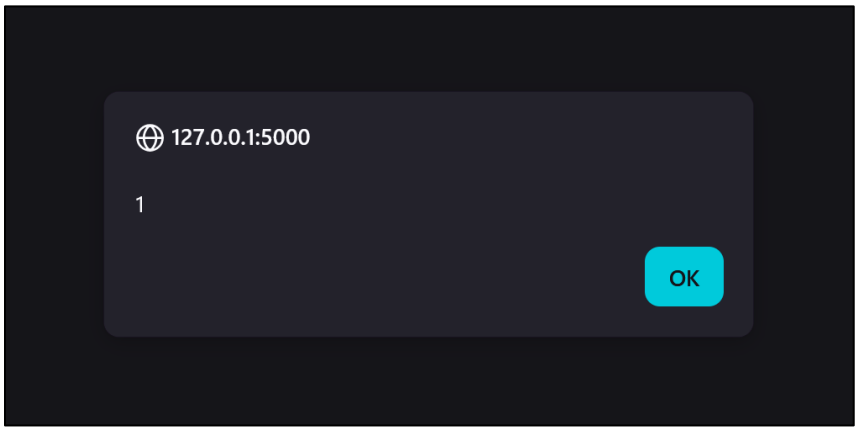
```
@app.route("/login", methods=["GET", "POST"])
def login():
    error = None
    next_url = request.values.get("next", url_for("products"))
    if request.method == "POST":
        username = request.form.get("username", "")
        password = request.form.get("password", "")
        query = """
        SELECT username
        FROM accounts
        WHERE username = ? AND password = ?
        LIMIT 1
    """
```



2. Reflected XSS:

Malicious input comes in with the request and is immediately reflected in the response.

In this application, the product search field is vulnerable.



Problem: Markup() treats input as already-safe HTML, So if q contains HTML/JavaScript, it isn't escaped.

```
def products():
    q = request.args.get("q", "")
    search_summary = None
    if q:
        rows = db().execute(
            "SELECT * FROM products WHERE name LIKE ?", (f"%{q}%",)
        ).fetchall()
        search_summary = Markup(f"Results for <mark>{q}</mark>")
    else:
        rows = db().execute("SELECT * FROM products").fetchall()
```

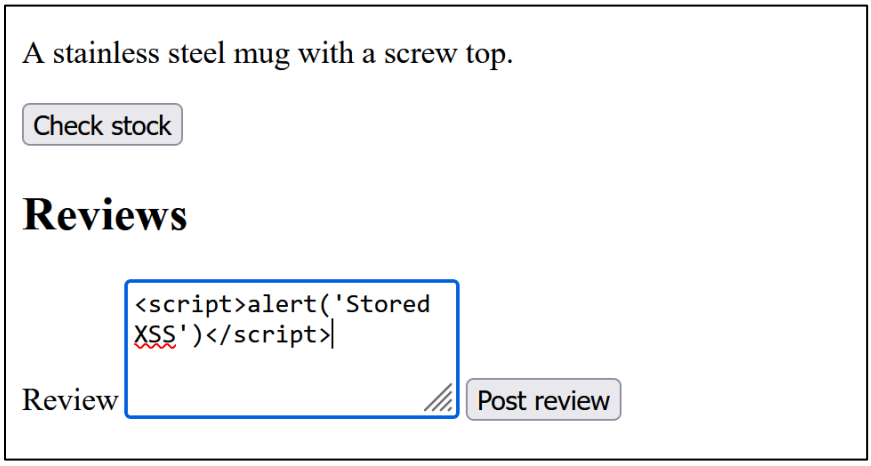
Fix: removing Markup() and allowing Flask/Jinja2 to automatically escape the user input before displaying it, so HTML and JavaScript entered by the user are treated as text instead of being executed.

```python
def products():
    q = request.args.get("q", "")
    search_summary = None
    if q:
        rows = db().execute(
            "SELECT * FROM products WHERE name LIKE ?", (f"%{q}%",)
        ).fetchall()
        if SECURITY_MODE:
            search_summary = f"Results for {q}"
        else:
            search_summary = Markup(
                f"Results for <mark>{q}</mark>"
            )
    else:
        rows = db().execute("SELECT * FROM products").fetchall()
```

3. Stored XSS:

The database stores the attacker's input.

In this application, the review field is vulnerable.



If we refresh the product page, the JavaScript executes again because the payload was stored in SQLite.

Problem: Markup() treats input as already-safe HTML, not escaped.

```
reviews = [Markup(row["body"].replace("\n", "<br>")) for row in review_rows]
```

Fix: removing Markup() when displaying the reviews and allowing Jinja2 to automatically escape the stored user input, so malicious HTML or JavaScript stored in the database is displayed as text instead of being executed.

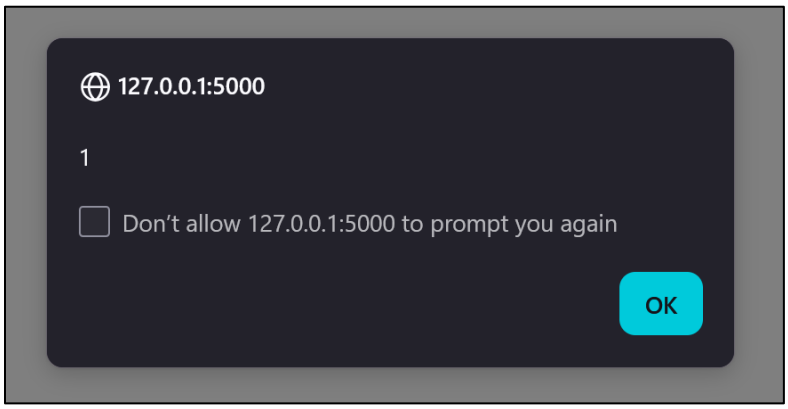
```python
def product_page(product_id, stock=None):
    product = get_product(product_id)
    review_rows = db().execute(
        "SELECT * FROM reviews WHERE product_id = ? ORDER BY id DESC", (product_id,)
    ).fetchall()
    if SECURITY_MODE:
        reviews = [
            escape(row["body"]).replace("\n", "<br>")
            for row in review_rows
        ]
    else:
        reviews = [
        Markup(row["body"].replace("\n", "<br>"))
        for row in review_rows
        ]
```

Not executed.

4. DOM-Based XSS:

The vulnerability happens when malicious input is taken from the URL and processed by JavaScript in the browser.

In this application, the message parameter in the product page is vulnerable.



Problem: the JavaScript gets the message value directly from the URL and inserts it into the page using innerHTML. Since innerHTML interprets HTML, malicious HTML/JavaScript can be executed.

```
const message = document.querySelector("#message");
const text = new URLSearchParams(location.search).get("message");

if (message && text) {
  message.innerHTML = text;
}
```

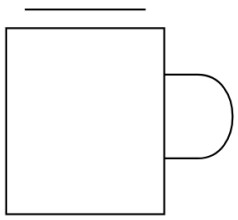
Fix: replacing innerHTML with textContent, so the user input is treated as plain text instead of being interpreted as HTML.

```
const text = new URLSearchParams(location.search).get("message");

if (message && text) {
    message.textContent = text;
}
```

Products

<img src=x onerror=alert(1)>

Travel mug



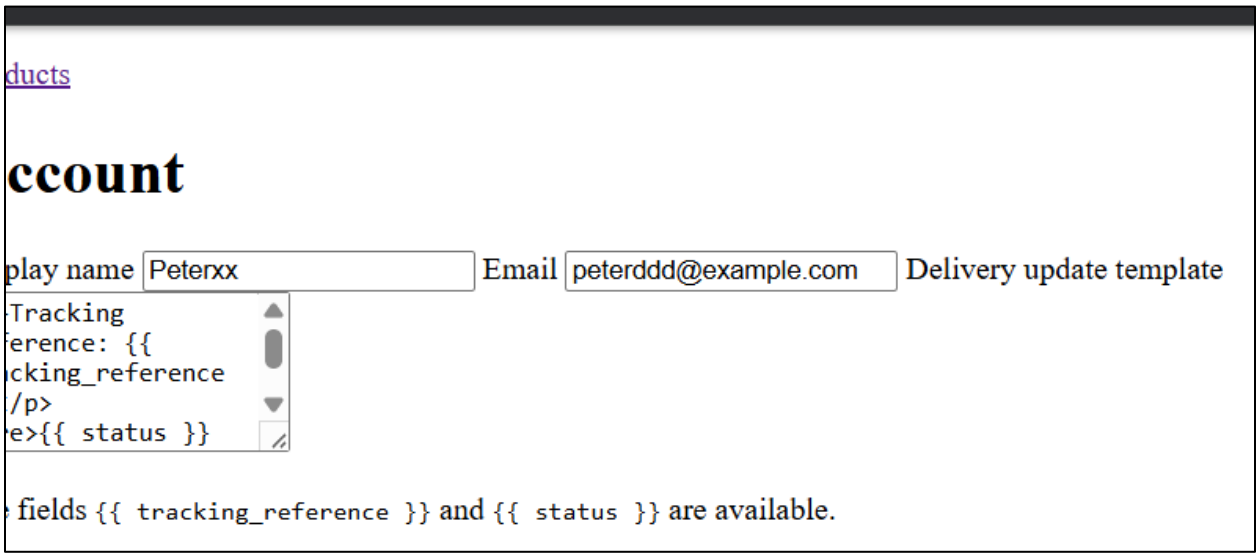
A stainless steel mug with a screw top.

Not executed.

5. CSRF:

Cross-Site Request Forgery is a vulnerability where an attacker tricks a logged-in user's browser into sending an unwanted request to the website.

In this application, the account update function is vulnerable.

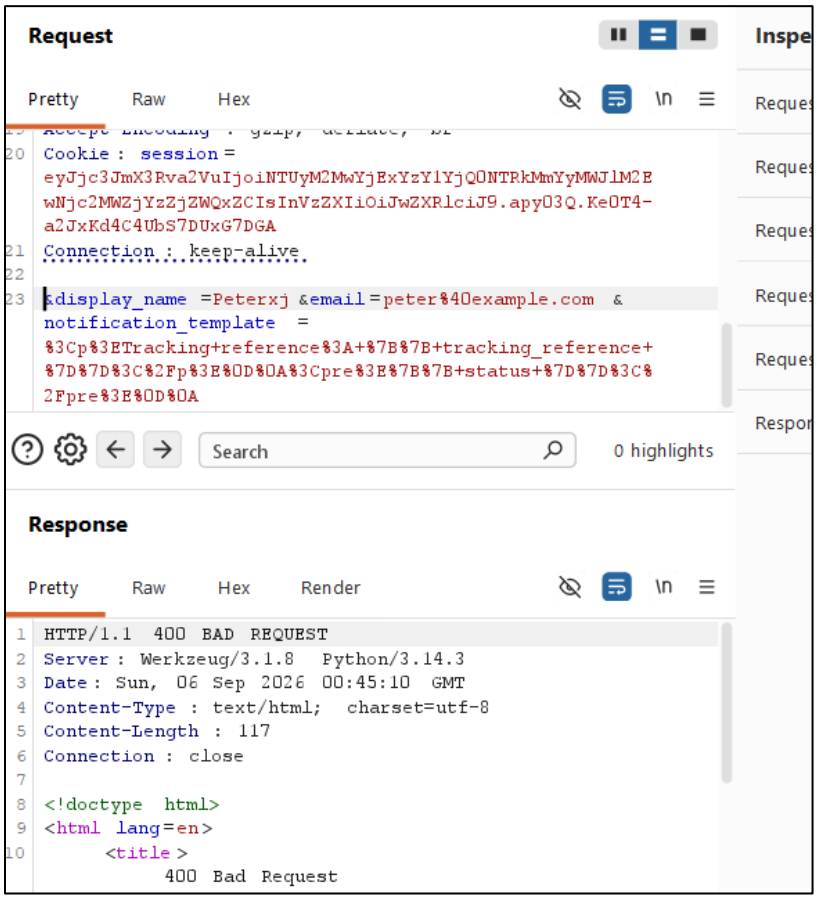


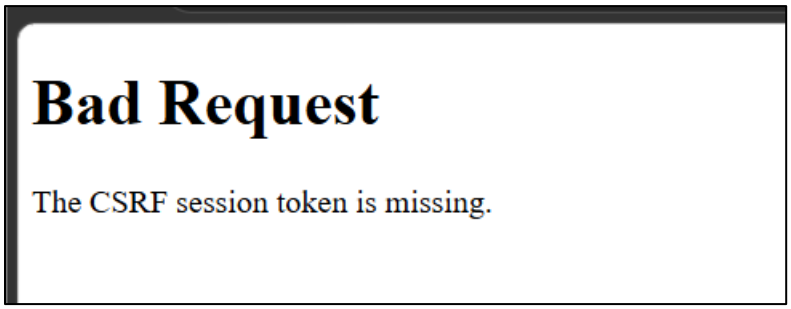
Problem: There is no CSRF token. The account form accepts POST requests without verifying that the request was actually made by the user.

```
<form method="post">
  <label>Display name <input name="display_name" value="Peter"></label>
  <label>Email <input name="email" value="peter@example.com"></label>
  <label>
```

Fix: adding CSRF protection using Flask-WTF and requiring a valid CSRF token with state-changing requests, so forged requests from another website are rejected.

```
<h1>Account</h1>

<form method="post">

  <input type="hidden" name="csrf_token" value="IjY5NTg0NTg4YWZjNjZmNjMwOTU2ZDdmMmE2MzczYTgzZWQ3MzU0Nzgi.ap71cQ.9issLqbtrOe1OLJ5tqzE1YGyM4w">
```





6. SSRF:

Server-Side Request Forgery is a vulnerability where an attacker can make the server send requests to a URL chosen by the attacker.

In this application, the stock-check function is vulnerable.

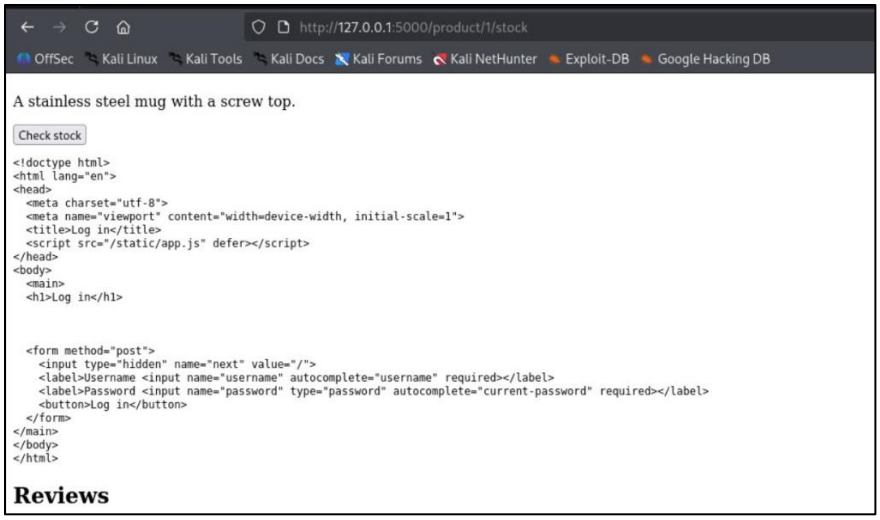
```
<form action="/product/1/stock" method="post">
  <input type="hidden" name="stock_api" value="http://127.0.0.1:5001/stock/1">
  <button>Check stock</button>
</form>
```

Problem: the application accepts the stock_api URL from the user and uses urlopen() to make a request without checking whether the destination is allowed. This allows the server to access internal services such as the supplier service.

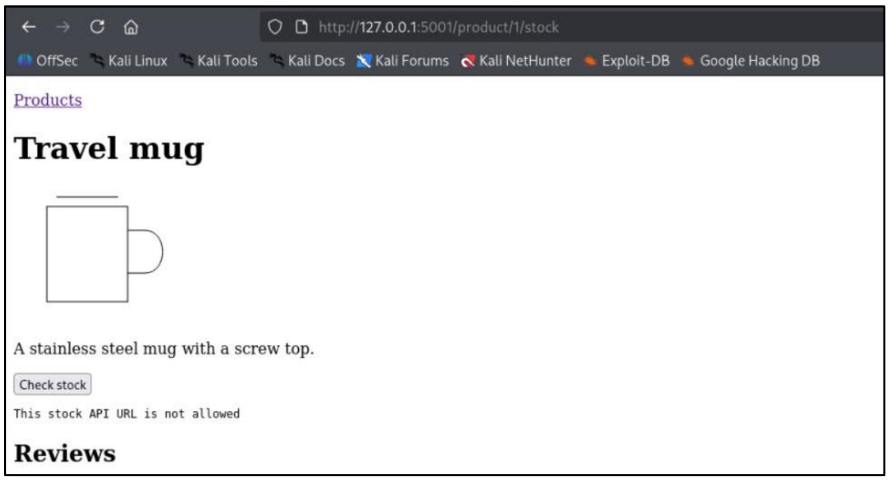
```
def check_stock(product_id):
    try:
        with urlopen(request.form.get("stock_api", ""), timeout=4) as response:
            stock = response.read(12000).decode("utf-8", errors="replace")
    except Exception as error:
        stock = str(error)
    return product_page(product_id, stock)
```

Test: The stock_api parameter was changed to http://127.0.0.1:5001/config before submitting the stock request.

Result: The main server fetched the internal supplier service and returned its response, demonstrating that the server can be forced to make requests to an attacker-controlled destination.



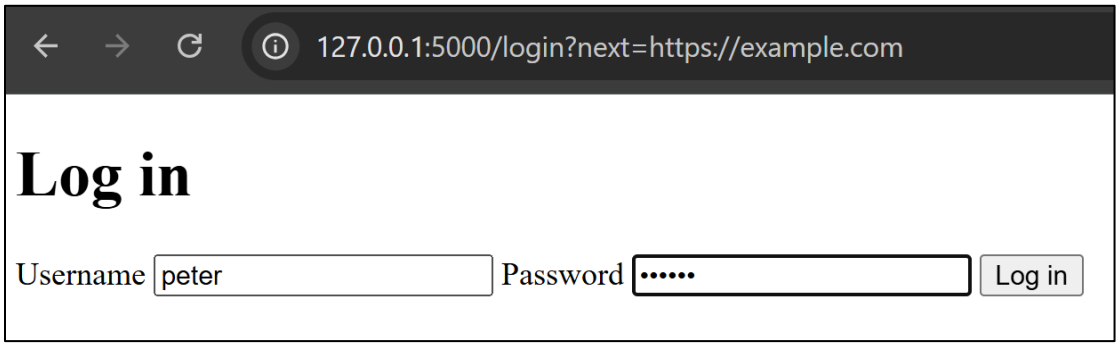
Fix: validating the requested URL and only allowing requests to trusted/allowed destinations, preventing the server from accessing unauthorized internal resources.

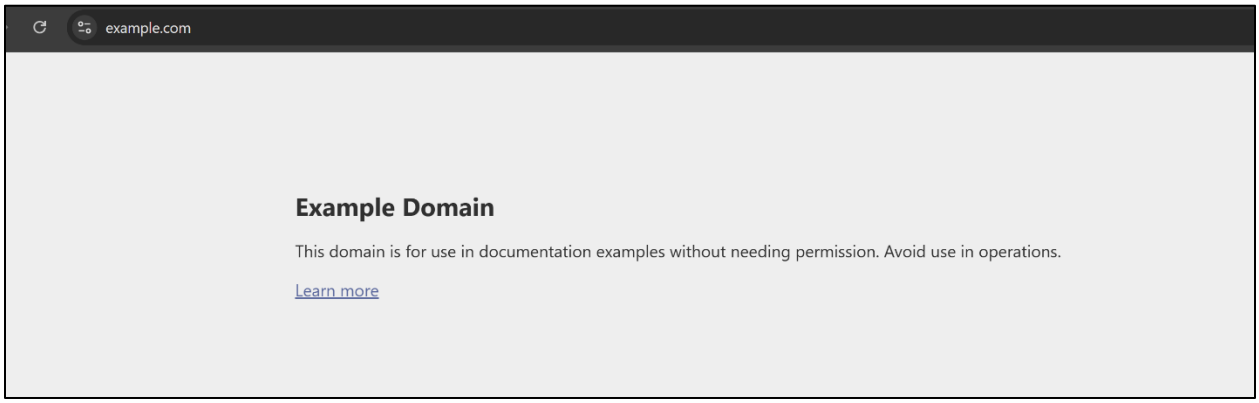


7. Open Redirect:

An Open Redirect is a vulnerability where an attacker can control the URL that the application redirects a user to.

In this application, the login function is vulnerable.





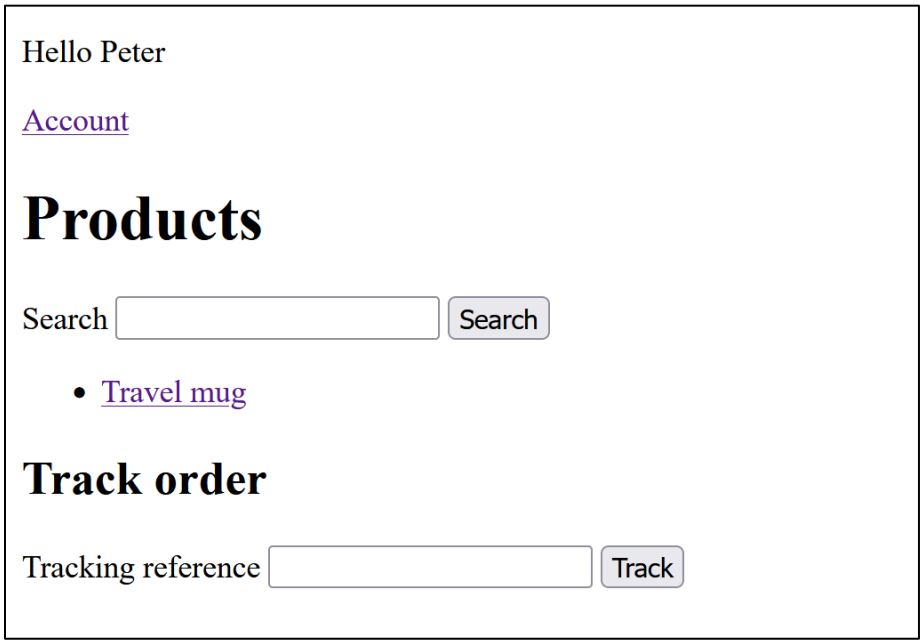
Problem: the next parameter is taken directly from the request and passed to redirect(), allowing an attacker to provide an external URL.

```
def login():
    error = None
    next_url = request.values.get("next", url_for("products"))
    if request.method == "POST":
        username = request.form.get("username", "")
        password = request.form.get("password", "")
```

Fix: validating the next URL before redirecting and only allowing safe internal destinations, preventing users from being redirected to untrusted websites.

```
if user:
    session["user"] = user["username"]

    if is_safe_url(next_url):
        return redirect(next_url)

    return redirect(url_for("products"))
error = "That login did not work."
```

Now After we log in:



No redirection.

8. SSTI:

Server-Side Template Injection is a vulnerability where user-controlled input is treated as a server-side template and evaluated by the template engine.

In this application, the template in the product page is vulnerable.

Problem: the notification template is stored as user input and later passed directly to render_template_string(). This causes template expressions entered by the user to be evaluated by Jinja2.

```
status = result.stdout + result.stderr or "No matching order."
source = get_notification_template(session["user"])
tracking_result = Markup(
    render_template_string(
        source,
        tracking_reference=tracking,
        status=status,
    )
```

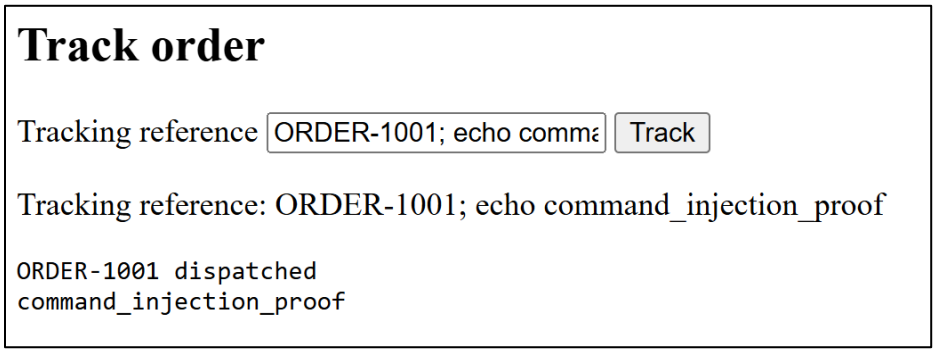
Fix: treating the notification template as normal user data instead of passing it directly to render_template_string(), preventing user input from being interpreted as a server-side template.

```
status = result.stdout + result.stderr or "No matching order."
source = get_notification_template(session["user"])
if SECURITY_MODE:
    tracking_result = render_template(
        "tracking_result.html",
        tracking_reference=tracking,
        status=status,
    )
else:
    tracking_result = Markup(
        render_template_string(
            source,
            tracking_reference=tracking,
               status=status,
        )
    )
tracking_result = Markup(
    str(Markup.escape(source)).replace(
        "{{ tracking_reference }}", str(Markup.escape(tracking))
    ).replace(
        "{{ status }}", str(Markup.escape(status))
    )
)
```

9. Command Injection:

Command Injection is a vulnerability where an attacker can execute additional operating system commands by inserting them into input that is passed to a shell.

In this application, the order tracking function is vulnerable.



Problem: the tracking reference is directly concatenated into a shell command and executed using shell=True. This allows shell characters in the input to change the command being executed.

```
tracking = request.args.get("tracking")
tracking_result = None
if tracking is not None:
    try:
        result = subprocess.run(
            "bash " + str(CARRIER_LOOKUP) + " " + tracking,
            shell=True,
            input="",
            capture_output=True,
            text=True,
            timeout=4,
        )
```

Fix: removing shell=True and passing the tracking reference as a separate argument to subprocess, while also validating that the tracking reference follows the expected format.

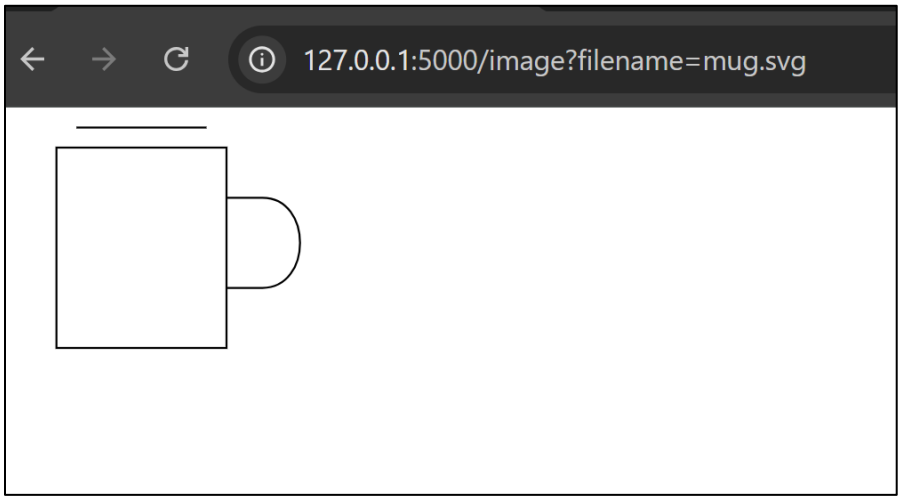
```
tracking = request.args.get("tracking")
tracking_result = None
if tracking is not None:
    try:
        if not TRACKING_PATTERN.fullmatch(tracking):
            return "Invalid tracking reference", 400

        result = subprocess.run(
            ["bash", str(CARRIER_LOOKUP), tracking],
            capture_output=True,
            text=True,
            timeout=4,
```

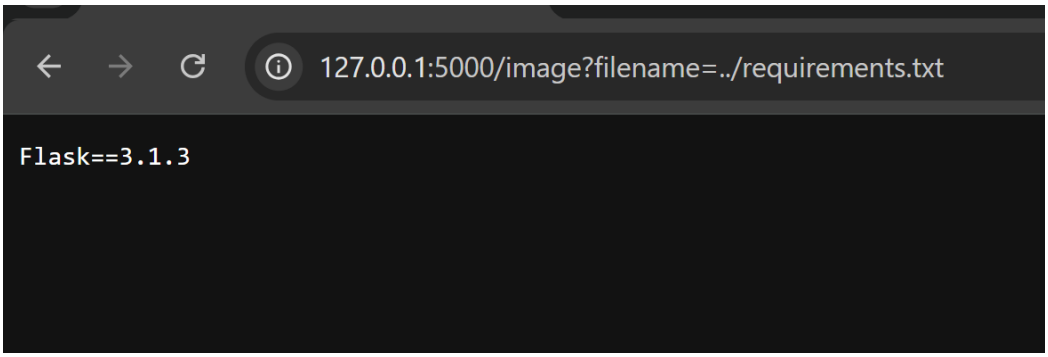
Invalid tracking reference

10. Path Traversal:

Path Traversal is a vulnerability where an attacker can use path manipulation sequences such as ../ to access files outside the intended directory.

In this application, the image function is vulnerable.





Problem: the filename is taken directly from the request and added to the images directory path without checking whether the resulting path stays inside the intended directory.

```
def image():
    return send_file(ROOT / "images" / request.args.get("filename", ""))
```

Fix: validating the requested file path and ensuring that it stays inside the images directory before allowing the file to be accessed.

```python
def image():
    filename = request.args.get("filename", "")

    if SECURITY_MODE:
        filename = secure_filename(filename)

        if not filename:
            return "Invalid filename", 400

        allowed_extensions = {
            ".png", ".jpg", ".jpeg",
            ".gif", ".svg", ".webp"
        }

        if Path(filename).suffix.lower() not in allowed_extensions:
            return "File type not allowed", 403

        image_root = (ROOT / "images").resolve()
        image_path = (image_root / filename).resolve()

        if image_root not in image_path.parents:
            return "Invalid file path", 403

        return send_file(image_path)
```